

ESTRUCTURAS DE DATOS

Ayudantía 1: C++

Ignacia Nettle Oliveri
inettle@usm.cl

BLOQUE 1

Fundamentos

Estructura básica, compilación y ejecución, variables y tipos de datos, lectura y escritura en Terminal y operadores básicos.

Estructura Básica de un Programa en C++

```
#include <iostream>
using namespace std;

int x = 10;
int example(int num){ return num+x; }

int main(){
    int a = 1;
    int b = example(a);
    cout << example(b); << endl;
    return 0;
}
```

- ▷ **Linking section**: contiene *header files* y librerías necesarias para la correcta ejecución del programa.
- ▷ **Namespaces**: organiza identificadores para prevenir conflictos cuando dos entidades de distintas librerías usan el mismo nombre. `namespace std` permite no tener que escribir `std::` cuando se quiera usar `cin` o `cout`, por ejemplo.
- ▷ **Global and Function Declaration**: contiene variables visibles y accesibles por la totalidad del programa y funciones definidas por el usuario.
- ▷ **Main function**: sección **fundamental**. Ejecución del programa siempre empieza en esta función.
- ▷ `return 0`: indica al Sistema Operativo que el programa se ejecutó exitosamente.

Al **finalizar** la ejecución del programa, ¿qué valores tienen a y b? ¿qué se imprime en la terminal?

Compilación y Ejecución

C++ es un lenguaje **compilado**, es decir, es necesario traducir el archivo a lenguaje de máquinas antes de ejecutarlo para que la máquina pueda leerlo fácil y rápidamente.

```
# 1. Compilación
g++ main.cpp -o main -Wall

#2. Ejecución
./main
```

- ▷ `-o` : se usa para especificar el nombre del ejecutable.
- ▷ `-Wall` : útil para ver los *warnings* del compilador. **No son errores**, pero es recomendable evitarlos pues pueden producir comportamiento inesperado.

Si se quiere compilar múltiples archivos en un solo ejecutable:

```
g++ file1.cpp file2.cpp file3.cpp -o main -Wall
```

Los header files **no se compilan**.

Variables

Las variables son contenedores que guardan datos. El proceso para crear una variable consiste en dos partes: **declaración** e **inicialización**.

- **Declaración:** consiste en especificar el tipo y nombre de la variable. El tipo de una variable es **immutable** tras ser declarada.
- **Inicialización:** consiste en asignarle un valor a dicha variable.

Declaración e inicialización pueden hacerse en una sola línea.

Tipos de Variables

Tipo	Ejemplos
Primitivas	<code>int</code> , <code>float</code> , <code>double</code> , <code>char</code> , <code>bool</code>
Derivadas	arreglos, punteros, referencias y funciones
User-defined	<code>class</code> , <code>struct</code> , <code>typedef</code> , <code>using</code>

Variables - Extras

Casting

Convierte temporalmente el tipo de una variable a otro.

```
int a = 5, b = 2;

// División entera: res1 = 2
double res1 = a/b;

// Casting, res2 = 2.5
double res2 = double(a)/double(b);
```

Sizeof

Retorna el tamaño exacto en **bytes** que ocupa un tipo de variable en memoria:

```
cout << sizeof(int) << endl; // Típicamente 4 bytes
cout << sizeof(double) << endl; // Típicamente 8 bytes
cout << sizeof(char) << endl; // Siempre 1 byte
```

Lectura y Escritura: Terminal

Flujo de datos

Los datos se leen y escriben usando **streams**, es decir, secuencias de bytes.

- ▶ **Input stream**: datos fluyen desde una fuente de entrada hacia la memoria de la máquina.
- ▶ **Output stream**: datos fluyen desde la memoria de la máquina hacia una fuente de salida.

Estos streams están definidos en la librería `<iostream>`, por lo que es fundamental que siempre esté incluida en los programas.

Flujos más usados

Los objetos de stream más comunes son **cin** para leer input y **cout** para imprimir en terminal.

cin usa el operador `>>` para extraer la información del input stream y para luego guardarla en una variable especificada. **cout** usa el operador `<<` para imprimir en la terminal un string a los datos almacenados en una variable.

```
int main(){  
    int x;  
    cin >> x;  
    cout << x << endl;  
}
```

Lectura y Escritura: Archivos

Flujo distinto

Para leer y escribir en archivos se usan los **streams** definidos en la librería `<fstream>`.

```
#include <fstream>
```

- `ifstream`: para **leer** un archivo.
- `ofstream`: para **escribir** en un archivo.
- `fstream`: para **leer y escribir** en un archivo.

Misma idea que `cin / cout`

Ya que los archivos también son *streams* de datos, los operadores `<<` y `>>` funcionan de la misma forma.

```
ifstream file_in("in.txt");  
ofstream file_out("out.txt"); // Si archivo no existe, lo crea  
  
cin >> x;  
file_in >> x;  
  
cout << x;  
file_out << x;  
  
file_in.close();  
file_out.close();
```

Tal y como en lectura y escritura en terminal, si se quiere leer una línea completa hasta el primer salto de línea, se usa `getline`.

Operadores Básicos en C++

Categoría	Operadores	Notas
Aritméticos	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	<code>/</code> entre enteros trunca los decimales.
Relacionales	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	Retornan <code>bool</code> (<code>true</code> / <code>false</code>).
Lógicos	<code>&&</code> (AND), <code> </code> (OR), <code>!</code> (NOT)	Diferentes a Python (<code>and</code> , <code>or</code> , <code>not</code>).
Incremento / Decremento	<code>++</code> , <code>--</code>	Existe prefijo (<code>++x</code>) y posfijo (<code>x++</code>).

BLOQUE 2

Estructuras de Control

`if` / `else`, `while`, `do-while` y `for`: cómo dirigir el flujo de un programa en C++.

Condicionales: `if/else if/else`

Decisiones Binarias

Ejecuta un bloque **si y solo si** la condición se cumple.

```
if(condition){  
    /* instrucción si condición se cumple */  
} else {  
    /* instrucción si condición no se cumple */  
}
```

Encadenar Alternativas

Con `else if` se evalúan condiciones en orden. Se ejecuta **sólo el primer** bloque que cumple la condición.

```
if(first_condition){  
    /* instrucción si primera condición se cumple */  
} else if(second_condition){  
    /* instrucción si segunda condición se cumple */  
} else {  
    /* instrucción "default" */  
}
```

Bucles: `while` vs `do-while`

`while`: "repetir mientras se cumpla"

Se evalúa la condición **antes** de cada iteración. Si no se cumple, cuerpo **no se ejecuta**.

```
while(condition){  
    /* code here */  
}
```

Un ciclo `while` puede no ejecutarse si la condición no se cumple desde un principio.

`do-while`: "al menos una ejecución"

La condición se evalúa **al final**. El cuerpo **siempre** se ejecuta al menos una vez.

```
do{  
    /* code here */  
} while(condition);
```

Generalmente, `while` / `do-while` se usan cuando **no se sabe de antemano** cuántas veces se repetirá el ciclo, o cuando la condición depende de input o cálculo.

Bucles: `for`

Estructura del `for`

Tres partes en la cabecera: inicialización, condición e incremento.

```
for(int i = 0; i < n; i++){  
    /* code here */  
}
```

- ▷ `int i = 0` : se ejecuta **una sola vez**, al entrar al ciclo.
- ▷ `i < n` : se evalúa **antes** de cada iteración.
- ▷ `i++` : se ejecuta **al final** de cada iteración.

Equivalente a `while`

```
int i = 0;  
while(i < n){  
    /* code here */  
    i++;  
}
```

- ▷ `for` es ideal cuando el rango y el comportamiento del contador son claros.
- ▷ El contador del `for` vive **sólo dentro del bucle**, no así el contador del `while`.

¿Cuál bucle elegir?

Estructura	Idea clave
<code>if</code> / <code>else</code>	Decidir entre caminos
<code>while</code>	Repetir si la condición ya es verdadera
<code>do-while</code>	Repetir y <i>después</i> preguntar
<code>for</code>	Iterar un rango conocido

BLOQUE 3

Punteros y Arreglos

Direcciones en memoria y colecciones contiguas.

¿Qué es un puntero?

Idea Clave

Un puntero es una variable que contiene **la dirección de memoria** en donde se almacena otra variable. Al igual que las variables normales, **también tienen un tipo definido**.

```
int x = 10;  
int *p = &x; // p "apunta" a x
```

- ▷ `&x` : dirección de `x`.
- ▷ `*p` : valor en esa dirección (*dereference*),
- ▷ `p` se puede reasignar a otra dirección **del mismo tipo**.

En la práctica

```
cout << x; // 10 (el valor)  
cout << &x; // 0x7ffe... (dirección)  
cout << p; // 0x7ffe... (dirección)  
cout << *p; // 10 (a lo que apunta p)
```

- ▷ `p` y `&x` son **la misma dirección**.
- ▷ `*p` y `x` son **el mismo valor** (mientras `p` apunte a `x`).
- ▷ `nullptr` permite inicializar un puntero que "no apunta a nada".
- ▷ Cuidado con usar `*p` si no estás seguro de que apunta a algo.

Arreglos: colección contigua

Características

Un arreglo guarda **múltiples elementos de un mismo tipo** uno tras otro en memoria. Son **homogéneos** y de largo **inmutable**.

```
int a[5] = {10, 20, 30, 40, 50};  
cout << a[0] << endl; // 10 (primer elemento)  
cout << a[4] << endl; // 50 (último elemento)
```

- Índices desde `0` hasta `n-1`
- Cuidado con salirse del rango: `arr[5]` o `arr[-1]` → Segmentation Fault.

Relación Arreglo - Puntero

Generalmente, el nombre de un arreglo se convierte en un **puntero** al primer elemento.

```
int a[5] = {10, 20, 30, 40, 50};  
int *p = a; // equivalente a &a[0]  
  
cout << *p << endl; // 10  
cout << *(p+1) << endl; // 20  
cout << p[1] << endl; // 20
```

BLOQUE 4

Funciones

Formato, parámetros y las distintas formas de pasar datos: por valor, por referencia y por puntero.

Estrutura típica

```
int suma(int a, int b){  
    return a+b;  
}
```

- ▷ `int` : tipo de retorno. En caso de no retornar nada, se usa `void`.
- ▷ `suma` : nombre de la función, identificador con el cuál se llamará.
- ▷ `(int a, int b)` : parámetros (tipo + nombre).
- ▷ `return` : devuelve a quien llamó la función un dato del tipo especificado.

El orden importa

El compilador lee el archivo de **arriba hacia abajo**: debe *conocer* una función antes de que esta sea llamada.

```
int main(){  
    cout << suma(3, 2) << endl;  
}  
  
int suma(int a, int b){  
    return a + b;  
}
```

Fallará. El compilador al momento de leer el `main` no sabe qué hace la función `suma`.

Paso por valor

Parámetro vs Argumento

- ▷ **Parámetro**: variable declarada en la función.
- ▷ **Argumento**: valor concreto que se le da a la función al llamarla.

```
int suma(int a, int b){ // a y b son parámetros
    return a + b;
}

int main(){
    int x = 5, y = 3;
    cout << suma(x, y) << endl; // x e y son argumentos
}
```

Copia del valor original

Por defecto, C++ **copia** el argumento en el parámetro, por lo que cualquier modificación a esa variable **no tendrá efecto** fuera de la función.

```
void modificar(int n){
    n = 100;
}

int main(){
    int x = 50;
    modificar(x);
    cout << x << endl; // 50 - variable no cambió
}
```

Paso por referencia

Un *alias* del original

Usando `&` para definir un parámetro, estamos accediendo **directamente al valor de la variable**, no a una copia de ella. Esto permite que los cambios hechos a la variable dentro de la función se vea reflejada fuera de ella.

```
void duplicar(int &n){
    n = n*2;
}

int main(){
    int x = 5;
    duplicar(x);
    cout << x << endl; // 10
}
```

Referencia constante

Es posible evitar la copia de la variable y **prohibir** la modificación del mismo.

```
void imprimir(const string &texto){
    cout << texto << endl;
}
```

- ▷ Evitar la copia es más eficiente en términos de **memoria**.
- ▷ `const` deja claro que la variable está en "*modo lectura*".

Paso por puntero

El paso por puntero cumple la **misma idea** que el paso por referencia, pero pasando explícitamente una **dirección**.

```
void duplicar(int *n){
    *
    n = *n * 2;
}

int main(){
    int x = 5;
    duplicar(&x);
    cout << x << endl; // 10
}
```

Referencia vs Puntero

- ▶ Usando el paso por *referencia*, nos aseguramos que **siempre** usemos un valor válido. El puntero puede ser `nullptr`.
- ▶ El paso por referencia es más fácil de entender, puesto que se comporta igual que el paso por valor.
- ▶ El paso por puntero permite el "no hay valor" como una opción válida.

```
void f(int *x){
    if(x == nullptr) return;
    *x = 1;
}
```

Los arreglos **siempre** se pasan como punteros, por lo que cualquier modificación inevitablemente afecta al arreglo original. `a[]`, `a[5]` y `*a` son declaraciones **equivalentes** para el parámetro. Notar que el tamaño del arreglo **siempre** es un parámetro adicional.

BLOQUE 5

Memoria

Dónde viven las variables: estática, stack y heap.

Los tres tipos de memoria

Tipo	Qué guarda	Vive mientras	Quién la administra
Estática	Variables globales y <code>static</code>	Todo el programa	El compilador
Stack	Variables locales y parámetros	Dura la función que las creó	Automático (al entrar / salir)
Heap	Lo que pides con <code>new</code>	Hasta que hagas <code>delete</code>	Usuario

Memoria dinámica: `new` y `delete`

Pedir y Liberar

Sirve cuando **no se tiene seguridad en tiempo de compilación** de la memoria que se necesitará.

```
int *p = new int; // reserva memoria en el heap
*p = 42;
/* code here */
delete p; // libera dicha memoria
p = nullptr; // buena práctica
```

- ▷ `new` retorna un **puntero** a la memoria reservada.
- ▷ Cada `new` necesita su propio `delete`.

Arreglos dinámicos

```
int n;
cin >> n; // tamaño del arreglo decidido en ejecución.
int *a = new int[n];
/* code here */
delete[] a;
```

- ▷ Cuando se trabaja con arreglos, se usa `new type[size]` y `delete[]` para pedir y liberar memoria.

Memoria dinámica: Errores clásicos

Memory Leak

Los **memory leaks** ocurren al pedir memoria y **nunca liberarla**.

- ▶ El bloque sigue reservado, pero es inaccesible por el resto del programa.
- ▶ Si bien en programas pequeños las consecuencias son casi imperceptibles, en programas grandes, los *leaks* se acumulan, causando que el programa necesite demasiada memoria para ejecutarse.
- ▶ Es recomendado usar `valgrind` para monitorear el correcto uso de memoria.

Segmentation Fault

Ocorre cuando se quiere acceder a bloques de memoria que ya fueron liberados.

```
int *p = new int[5];  
/* code here */  
delete[] p;  
cout << p[2] << endl; // segmentation fault
```

- ▶ Cuidado con usar `delete` dos veces sobre el mismo puntero

BLOQUE 5

Análisis de Algoritmos

Evaluación de eficiencia de programas y algoritmos.

Uso eficiente de recursos

¿Qué es un algoritmo?

Un algoritmo es un conjunto de **instrucciones** bien definidas y ordenadas que permiten realizar una tarea mediante **pasos sucesivos**.

- ▶ Un algoritmo tiene una **entrada**, es decir, el número de datos a procesar.
- ▶ Un algoritmo produce al menos una **salida** luego de completar su ejecución.

¿Por qué se estudia?

Un buen algoritmo, además de resolver el problema al que se enfrenta, debe hacer un **uso eficiente** de los recursos computacionales disponibles. Para efectos de este curso, nos enfocaremos principalmente en **Tiempo de ejecución** y cómo esta crece en proporción al tamaño de la entrada.

- ▶ Si **n** es el tamaño de entrada del algoritmo, **$T(n)$** es su tiempo de ejecución.

Reglas para calcular Tiempos de Ejecución

Expresión	Tiempo de Ejecución
Operaciones Básicas	1 Unidad de Tiempo
Secuencia de Sentencias	Suma de los tiempos de cada sentencia
Condicionales	$T(\text{evaluar la condición}) + \max(T(\text{Secuencia-If}), T(\text{Secuencia-Else}))$
Bucles	$T(\text{evaluar la condición}) + \#iteraciones * (T(\text{Secuencia-While}) + T(\text{evaluar condición}))$

En un bucle siempre se evaluará la condición **al menos una vez**.

Las declaraciones **no cuestan unidades de tiempo**.

Ejemplo

¿Cuánto es el Tiempo de Ejecución?

```
int n;  
cin >> n;  
  
int *a = new int[n];  
  
for(int i = 0; i < n; i++){  
    cin >> a[i];  
}  
  
for(int i = 0; i < n; i++){  
    for(int j = 0; j < 5; j++){  
        cout << a[i]*j << endl;  
    }  
}  
delete[] a;
```

Puntos clave

- ▷ ¿Cuántas iteraciones se realizan?
 - ▷ ¿La cantidad de iteraciones dependen del tamaño de la entrada o son constantes?
- ▷ ¿Cuánto cuestan la secuencia de sentencias de los bucles?
- ▷ Una sola línea puede contener **múltiples operaciones básicas**.

Ejemplo

¿Cuánto es el Tiempo de Ejecución?

```
int n;  
cin >> n;  
  
int *a = new int[n];  
  
for(int i = 0; i < n; i++){  
    cin >> a[i];  
}  
  
for(int i = 0; i < n; i++){  
    for(int j = 0; j < 5; j++){  
        cout << a[i]*j << endl;  
    }  
}  
  
delete[] a;
```

Solución - Secuencia Inicial

- ▷ `int n` : declaraciones cuestan 0 unidades de tiempo.
- ▷ `cin >> n` : (+1) verificar estado del stream, (+1) *stop and wait* por el input, (+1) leer el input, (+1) asignar el valor del input a `n`.
- ▷ `int *a = new int[n]` : (+1) leer el valor de `n`, (+1) llamar al alocador de la memoria *heap*, (+1) asignar la dirección de memoria al puntero `a`, (+1) escribir el tamaño de la memoria pedida en un *header* oculto para que el sistema sepa cuánta memoria necesita liberar. (+n) alocar los `n` *slots* de memoria en el heap. (+n) limpiar los `n` *slots* en caso de que hayan habido valores residuales de programas pasados.

$$\text{Initial Sequence Units} = 2n + 8$$

Ejemplo

¿Cuánto es el Tiempo de Ejecución?

```
int n;  
cin >> n;  
  
int *a = new int[n];  
  
for(int i = 0; i < n; i++){  
    cin >> a[i];  
}  
  
for(int i = 0; i < n; i++){  
    for(int j = 0; j < 5; j++){  
        cout << a[i]*j << endl;  
    }  
}  
  
delete[] a;
```

Solución - Primer Loop

- ▷ **Loop Header**: (+1) asignación de `i`, (+1) comparación inicial `i < n`.
- ▷ **Secuencia del Loop**: (+1) comparación `i < n`, (+3) *stop and wait*, leer input, asignarlo a `a[i]`, (+1) leer `i`, (+1) calcular la dirección de `a[i]`, (+1) incremento de `i`. **Todo esto `n` veces.**

$$\textit{First Loop Units} = 7n + 2$$

Notar que la verificación del Input Stream **se hace sólo una vez** para inputs consecutivos.

Ejemplo

¿Cuánto es el Tiempo de Ejecución?

```
int n;
cin >> n;

int *a = new int[n];

for(int i = 0; i < n; i++){
    cin >> a[i];
}

for(int i = 0; i < n; i++){
    for(int j = 0; j < 5; j++){
        cout << a[i]*j << endl;
    }
}

delete[] a;
```

Solución - Segundo Loop

- ▷ **Header loop externo**: (+1) asignación de `i`, (+1) comparación inicial de `i < n`.
- ▷ **Secuencia de loop externo**: (+1) comparación de `i < n`, (+1) incremento de `i`, (+1) asignación de `j`, (+1) comparación inicial de `j < 5`, cuerpo de loop interno. **Todo esto `n` veces.**
- ▷ **Secuencia del loop interno**: (+1) comparación de `j < 5`, (+3) leer `i`, calcular la dirección de `a[i]` y leer `a[i]`, (+1) leer `j`, (+1) multiplicación, (+1) escritura en terminal, (+2) `endl` escritura de `\n` y vaciar el *buffer* de escritura, (+1) incremento de `j`. **Todo esto `5` veces.**

$$\text{Second Loop Units} = 2 + 54n$$

Ejemplo

¿Cuánto es el Tiempo de Ejecución?

```
int n;  
cin >> n;  
  
int *a = new int[n];  
  
for(int i = 0; i < n; i++){  
    cin >> a[i];  
}  
  
for(int i = 0; i < n; i++){  
    for(int j = 0; j < 5; j++){  
        cout << a[i]*j << endl;  
    }  
}  
  
delete[] a;
```

Solución - Liberación de Memoria

- ▷ `delete[] a` : (+1) lectura del puntero, (+1) lectura del *header oculto* para saber el tamaño del bloque de memoria a liberar, (+1) devolver el bloque de memoria de *heap* a la memoria del sistema operativo.

$$\text{Memory Free Units} = 3$$

Solución - Total

$$T(n) = 63n + 15$$

Análisis Asintótico

Notación Big-O

La notación Big-O es una forma de expresar la **cota superior** de la complejidad temporal de un algoritmo, es decir, nos enfocaremos en el **peor caso posible**.

Una forma rápida de encontrar la notación Big-O del tiempo de ejecución de un algoritmo es:

- ▶ Ignorar los términos de $T(n)$ pequeños y quedarse sólo con los máximos.
- ▶ Ignorar la constante asociada al término de máximo orden.

Por ejemplo, la notación Big-O del ejemplo anterior sería $O(n)$.

Propiedad Importante

Cuando se tienen términos más complejos, puede que no sea tan sencillo determinar cuál es de orden superior. Para ello, usaremos la siguiente fórmula:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k$$

Si $k \neq 0$ o $k \neq \infty$, f y g tienen el **mismo orden**.
Si $k = 0 \Rightarrow f$ está incluido dentro del orden de g .

Análisis Asintótico

w: 250 h: 500

Ejemplo

Si se tienen dos términos, n y n^2 y se quiere saber cuál de los dos tiene orden mayor, podemos usar la propiedad del límite de la notación Big-O:

$$\begin{aligned}\lim_{n \rightarrow \infty} \frac{n}{n^2} \\&= \lim_{n \rightarrow \infty} \frac{1}{n} \\&= \frac{1}{\infty} = 0\end{aligned}$$

$\Rightarrow n$ está incluido dentro de n^2 , por lo que este último es de orden superior.

Si el límite da ∞ es un indicador poco seguro de que el término del denominador está incluido dentro del término del numerador, por lo que conviene invertirlos y volver a resolver el límite.

BLOQUE 6

Ejercicios

Ejercicio 1

Escribe un programa en C++ que calcule la suma de los números pares e impares en un arreglo dado. Los resultados deben ser impresos en la consola.

Solución

```
#include <iostream>
using namespace std;

int main(){
    /*Supongamos que el arreglo es de largo n y está definido previamente*/
    int even = 0, odd = 0;
    for(int i = 0; i < n; i++){
        if(arr[i]%2 == 0){
            even += arr[i];
        } else {
            odd += arr[i];
        }
    }

    cout << "Suma de pares: " << even << endl;
    cout << "Suma de impares: " << odd << endl;
    return 0;
}
```

Ejercicio 2

Implementa las funciones `append` e `insert` tal que simulen el comportamiento de los métodos de Python del mismo nombre para un **arreglo de caracteres**. Recuerda hacer correcto uso de punteros y memoria dinámica.

Solución: **append**

```
char* append(char *original, string palabra, int n){
    int largo_p = palabra.length();
    int new_size = n + largo_p;
    char *new_p = new char[new_size];

    for(int i = 0; i < n; i++){
        new_p[i] = original[i];
    }

    for(int j = n; j < new_size; j++){
        new_p[j] = palabra[j-n];
    }

    return new_p;
}
```

Solución: **insert**

```
char* insert(char *og, string pal, int n, int idx){
    int len = pal.length();
    int new_len = n+len;
    char *new_a = new char[new_len];

    for(int i = 0; i < idx; i++){
        new_a[i] = og[i];
    }

    for(int j = idx; j < idx+len; j++){
        new_a[j] = pal[j - idx];
    }

    for(int k = idx+len; k < new_len; k++){
        new_a[k] = og[k-len];
    }

    return new_a;
}
```

CIERRE

¡Eso es todo 🧐!

En este [repositorio](#) podrán encontrar todo el material usado.
Mucho éxito programando en C++.